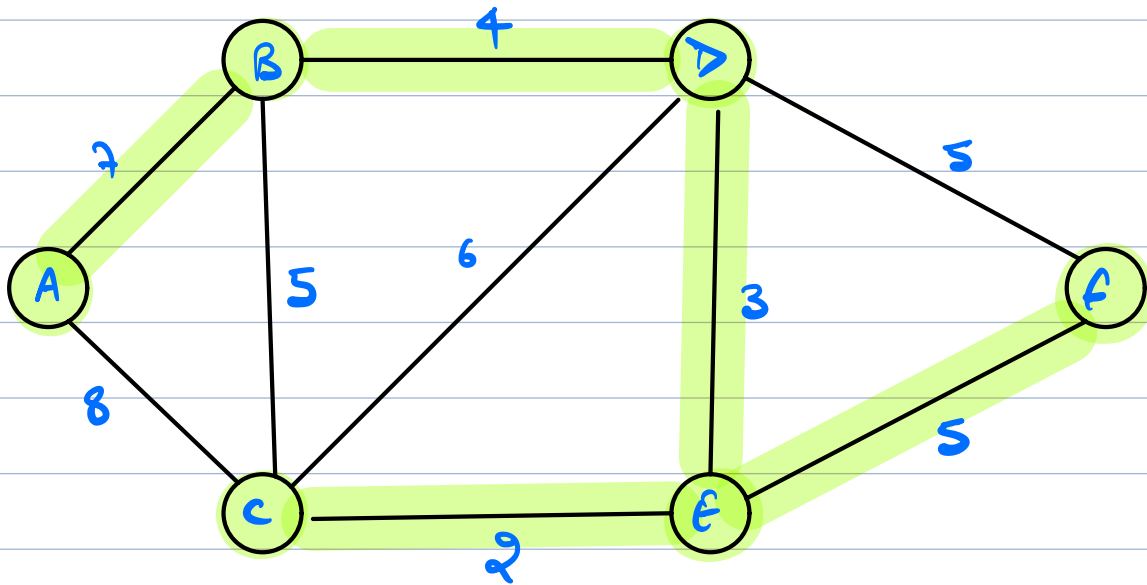


Spanning Tree

- Connect all nodes (vertices)
- Has no cycle
- Contain exactly $v-1$ edges

MST $\Rightarrow$ Spanning tree where sum of weights of edges selected is min.



vertices = { A, B, C, D, E, F }
edges = { A-B, B-D, D-E, E-C, E-F }

~~A-B, 7~~

~~A-C, 8~~

~~B-C, 5~~

~~B-D, 4~~

~~D-E, 3~~

A-B, 7

B-D, 4

D-E, 3

E-C, 2

B-C, 5

~~D - E, 3~~
~~D - F, 5~~
~~E - C, 2~~
~~E - F, 5~~

F - F, 5
D - F, 5
D - C, 6
A - C, 8

Prim's Algo

```
class Edge {  
    Node src;  
    Node dest;  
    int weight;
```

```
    int compare() {
```

```
    }
```

```
}
```

```
HashSet < Node > vst;  
MinHeap < Edge > edges;
```

```
list < Edge > mst;
```

```
vst.insert(src);
```

```
for (all Node u connected to src) {  
    edges.insert(new Edge(src, u, wsrc-u));  
}
```

```
while (! edges.isEmpty()) {
```

```
    Edge e = edges.getMin();
```

```
    if (! vst.contains (e.dest)) {
```

```
        Node d = e.dest;
```

```
        vst.insert (d);
```

```
        mst.insert (e);
```

```
        for (all Node u connected to d) {
```

```
            if (! vst.contains (u)) {
```

```
                edges.insert (new Edge(d, u, Wd-u));
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

MSTP / STP (Spanning Tree Protocol)

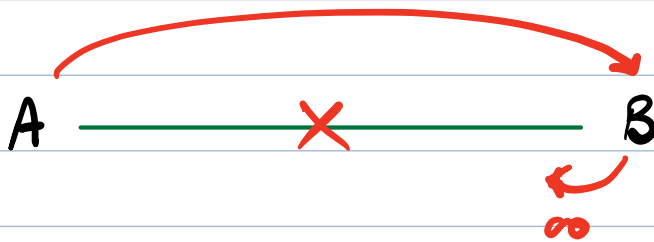
- ⇒ Used on layer 2 switches
- ⇒ Routers run a distributed MST algorithm.
- ⇒ Redundant links are logically blocked.
- ⇒ Results : A loop free topology.

Routing Protocol families

> Distance Vector Routing (Bellman Ford)

- Each router knows cost to its direct neighbours.
- Periodically a router broadcasts its entire routing table to neighbours.
- Neighbours update their routing tables based on received info.
- Converges over many rounds.

Problem: Count to infinity.



Protocols : RIP (max 15 hops)

: BGP

2) link State Routing (Dijkstra Based)

- Each router floods its local link state (neighbors + costs) to ALL routers
- Every router builds a complete topology map
- Every router independently runs dijkstra's algo to compute shortest paths

Protocols : OSPF
IS-IS

Comparison

Feature	Distance Vector	Link State
Information shared	Routing table (distances)	Link state (direct neighbors)
Algorithm basis	Bellman-Ford	Dijkstra
Topology knowledge	Partial (local view)	Full (global view)
Convergence speed	Slow	Fast
Bandwidth use	Low (periodic updates)	High (initial flooding)
Loop risk	Count-to-infinity	None (full topology)
Examples	RIP, BGP	OSPF, IS-IS

```

[Your Device]
  ↓ OSPF/IS-IS (inside your ISP's AS)
[Your ISP – AS 1234]
  ↓ BGP (between ASes)
[National ISP – AS 9829]
  ↓ BGP
[Global ISP – AS 6453]
  ↓ BGP
[Google – AS 15169]

```

- **Within an AS (Autonomous System):** OSPF or IS-IS (link-state)
- **Between ASes:** BGP (path-vector — shares AS path to prevent loops)

Routing v/s forwarding

Concept	Definition	When it happens	Plane
Routing	Deciding WHERE to send packets; building the routing table	Periodically (when topology changes)	Control plane
Forwarding	Actually moving packets out the correct interface	For every single packet	Data plane

- **Routing** is slow and intelligent (algorithms, protocols)
- **Forwarding** is fast and mechanical (table lookup, hardware ASICs)

What is a Router ??

Device with:

- Multiple network interfaces, each connected to a different network
- Maintains a routing table that maps

destinations to next hops.

- A routing engine to learn & maintain routes.
- A forwarding engine to process packets quickly.

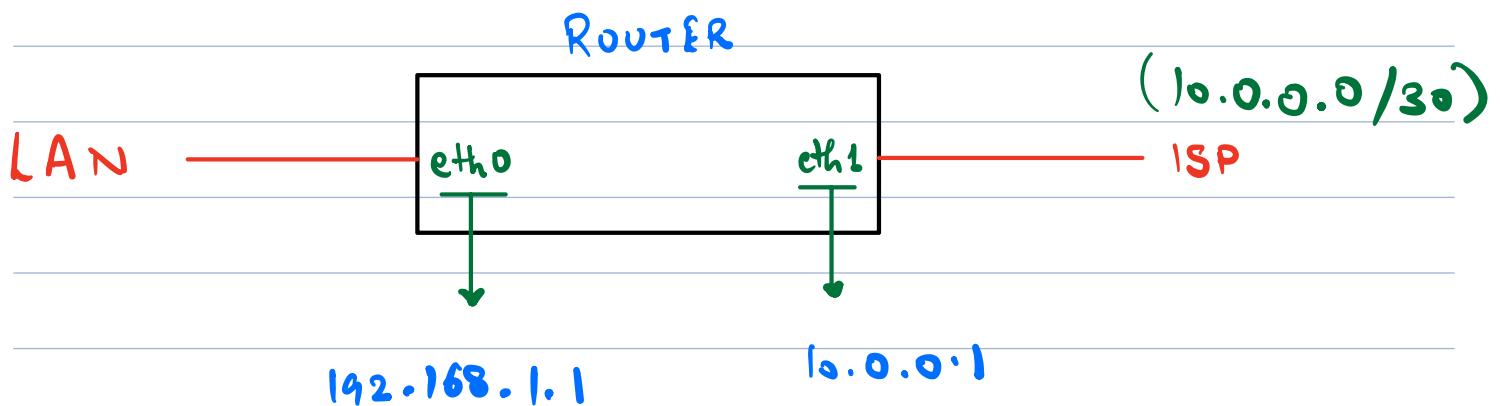
A network interface is a device's connection to one particular network.

eth0 → Ethernet interface

wlan0 → Wi-Fi interface

lo → Loopback interface

An IP address is assigned to a network interface.



LAN → [eth0 | ROUTER | eth1] → ISP
IN OUT

Interface eth0 — 192.168.1.1/24 (LAN)

|
[ROUTER] — |

|
Interface eth1 — 10.0.0.1/30 (WAN to ISP)

Routing Table

Routing Table Fields

Field	Description
Destination Network	The IP prefix this entry covers
Next Hop	IP of the next router to send the packet to
Interface	Which interface to send it out
Metric	Cost of this route (lower = more preferred)
Source	How the route was learned (connected, static, OSPF)